综合论文训练

AIoT 操作系统性能优化关键技术研究

系　　别：计算机科学与技术系

专　　业：计算机科学与技术

姓　　名：王　　铮

指导教师：陈　　渝　副教授

二〇二六年三月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定，即：学校有权保留论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

作者签名：　　导师签名：

日　　期：　　日　　期：

摘　要

【待完善】填写中文摘要

关键词：AIoT；操作系统；性能优化；ArceOS；Starry

Abstract

【待完善】 填写英文摘要

Keywords: AIoT; Operating System; Performance Optimization; ArceOS; Starry

目 录

第 1 章 绪论 1
1.1 研究背景 1
1.1.1 操作系统内核架构的演进 1
1.1.2 组件化操作系统与 ArceOS 1
1.1.3 Starry OS 的发展 1
1.2 相关工作 1
1.3 本文工作 1
第 2 章 系统设计 2
2.1 ArceOS 组件化基础 2
2.1.1 组件层次结构 2
2.1.2 Rust 组件化开发的挑战 2
2.1.3 crate_interface 机制 2
2.2 extern_trait 机制 2
2.2.1 设计动机 2
2.2.2 实现原理 2
2.2.3 应用场景 3
2.3 用户态切换机制 3
2.3.1 传统宏内核的问题 3
2.3.2 反转控制流设计 3
2.3.3 基于 axcpu 的实现 3
2.4 高效“内核-用户”交互设计 3
2.4.1 用户内存访问机制 3
2.4.2 缓冲区抽象设计 3
2.5 异步机制设计 4
2.5.1 设计目标 4
2.5.2 同步+异步混合架构 4
2.5.3 与信号和硬件的集成 4
2.6 整体组件架构 4
2.6.1 仓库拆分与解耦设计 4
2.6.2 组件复用与开源实践 4

第 3 章 系统实现 5
3.1 系统调用层 5
3.1.1 进程与任务管理 5
3.1.2 内存管理 5
3.1.3 其他系统调用 5
3.2 文件系统 5
3.3 网络、终端与其他子系统 6
3.4 信号机制 6
3.5 多级缓存优化 6
3.6 多核调度优化 6
3.7 虚拟内存优化 6
第 4 章 AI 应用支持 7
4.1 目标平台与硬件环境 7
4.2 NPU 驱动设计 7
4.3 AI 应用运行支持 7
第 5 章 实验评测 8
5.1 实验环境 8
5.2 功能测试 8
5.2.1 系统调用测例 8
5.2.2 应用兼容性测试 8
5.3 性能测试 8
5.3.1 系统调用开销 8
5.3.2 I/O 性能 9
5.4 开发工作量统计 9
第 6 章 总结与展望 10
6.1 工作总结 10
6.2 未来工作展望 10
参考文献 11
附录 A 外文资料的调研阅读报告（或书面翻译） 12
附录 B 其他附录内容 13
致 谢 14
声 明 15

插图清单

附表清单

符号和缩略语说明

AIoT	Artificial Intelligence of Things，人工智能物联网
OS	Operating System，操作系统

第 1 章 绪论

1.1 研究背景

1.1.1 操作系统内核架构的演进

【待完善】 简要介绍宏内核、微内核、Unikernel 三种架构的设计权衡（性能/隔离/灵活性），说明各自的适用场景与局限性。可配一张对比表格（参考 thuthesis-zyj 表 2.1）。1-2 段。

1.1.2 组件化操作系统与 ArceOS

【待完善】引出组件化设计理念，介绍 ArceOS 的灵活内核原则（One Architecture to build all）

1.1.3 Starry OS 的发展

【待完善】 梳理 Starry-Next → Starry Mix → StarryOS 主线的演进历程

1.2 相关工作

【待完善】 每类 2-3 句话即可：

- 其他 Rust OS：Asterinas（Framekernel）、Theseus、Tock
- Linux 兼容性内核：gVisor、Graphene/Occlum（TEE 场景）
- 性能优化相关：io_uring、CortenMM（SOSP'25 最佳论文）

1.3 本文工作

【待完善】 分 3-4 段简述本文主要贡献：

1. 组件化设计探索（extern_trait、用户态切换机制、30+仓库解耦）
2. 兼容性内核实现（200+ syscall、Alpine Linux 生态）
3. 性能优化（用户访存、缓冲区抽象、异步 I/O、多级缓存）
4. AI 应用支持方向（展望）

第 2 章 系统设计

【待完善】开头一段：说明本章在 ArceOS 已有组件化基础上，进一步探索宏内核场景下的关键设计问题，涵盖组件化机制、用户态切换，以及高效内核-用户交互的设计。

2.1 ArceOS 组件化基础

2.1.1 组件层次结构

【待完善】 介绍 ArceOS 的组件层次划分（axhal / axruntime / modules / crates / apps）， 配一张层次结构图（参考 thuthesis-zyj 图 3.1）。

2.1.2 Rust 组件化开发的挑战

【待完善】说明 Rust 依赖树在编译时必须 resolve 的特性，举 `#[panic_handler]`、`#[global_allocator]` 等典型反向依赖例子，说明这在 OS 开发中尤为突出。

2.1.3 crate_interface 机制

【待完善】简述 crate_interface 解决的问题（跨 crate 的接口定义与实现解耦），指出其局限性（不支持复杂类型传递、RAII 语义受限），引出 extern_trait 的动机。

2.2 extern_trait 机制

2.2.1 设计动机

【待完善】对比 `Box<dyn Trait>`、`*mut ()` 等现有方案的不足（overhead、内存安全、 需要动态分配），说明 extern_trait 的设计目标。

2.2.2 实现原理

【待完善】 说明 extern_trait 的核心思想：通过过程宏在编译期生成调用桩， 实现零开销的跨 crate 接口调用，支持任意类型和 RAII 语义。可附代码示例。

2.2.3 应用场景

【待完善】列举在 StarryOS 中的具体应用（TaskExt、on_enter/on_leave/drop 等），以及已发布/计划发布到 crates.io 的情况（extern-trait crate）。

2.3 用户态切换机制

2.3.1 传统宏内核的问题

【待完善】说明传统宏内核“用户程序调用内核”模式对组件化的影响：对用户程序流控制能力差，难以将控制流逻辑组件化（注册回调等方式的局限）。

2.3.2 反转控制流设计

【待完善】说明 StarryOS 的设计：内核代码“调用”用户程序，主动控制用户程序流， 对比 Asterinas 的类似思路，说明其对组件化架构的意义。

2.3.3 基于 axcpu 的实现

【待完善】简述 axcpu 组件的职责（封装用户态/内核态切换的底层机制），说明实现难点及关键设计决策。

2.4 高效“内核-用户”交互设计

2.4.1 用户内存访问机制

【待完善】说明内核访问用户内存的设计目标——零额外开销（无异常时）。介绍 Exception Fixup 机制：在汇编层面处理页异常并保证控制流不受破坏， 参考 Linux 的实现思路，在没有异常发生时不存在额外开销。

2.4.2 缓冲区抽象设计

【待完善】说明传统 `read/write` 接口（`&mut [u8]`）在处理用户数据时需要额外分配与拷贝的问题。介绍 Buf/BufMut 抽象：统一内核与用户缓冲区，大部分场景实现 one-copy I/O，说明其与 Linux 设计的契合性，以及更 Rusty 的接口特点。

2.5 异步机制设计

2.5.1 设计目标

【待完善】说明异步机制的目标：消除低效轮询，在 I/O 瓶颈场景下提升吞吐量。分析纯同步与纯异步实现各自的优缺点，引出混合方案的必要性。

2.5.2 同步+异步混合架构

【待完善】介绍 Waker/Poller 机制：PollSet（无锁单向链表存储 Waker）、Pollable 接口（poll + register），以及 Poller 实现的核心逻辑。 说明如何兼容 POSIX poll 语义。可附架构图。

2.5.3 与信号和硬件的集成

【待完善】 说明异步机制与信号（可中断的系统调用）的交互， 以及外设中断（VirtIO 网卡/输入、UART）通过注册 Waker 实现硬件唤醒的机制。

2.6 整体组件架构

2.6.1 仓库拆分与解耦设计

【待完善】 说明将 StarryOS 拆分为 30+ 个独立代码仓库的设计原则， 各仓库的功能边界，以及如何通过接口隔离保持可维护性。

2.6.2 组件复用与开源实践

【待完善】介绍已被其他团队复用的组件（如被 Starry X、undefined 等队伍使用），以及计划发布到 crates.io 的组件（weak-map、scope-local、extern-trait 等）。

第 3 章 系统实现

【待完善】开头一段：说明本章在第 2 章设计的基础上，介绍 StarryOS 兼容性内核的具体实现， 以及多级缓存、多核调度、虚拟内存等性能优化的实现情况。

3.1 系统调用层

【待完善】介绍系统调用的整体处理流程（用户态陷入 → 分发 → 处理 → 返回），以及如何在组件化架构下组织系统调用模块。可附处理流程图。

3.1.1 进程与任务管理

【待完善】简述 fork/exec/wait/exit/clone 等核心进程管理系统调用的实现要点，重点说明与 ArceOS 任务模型的对接方式，以及 TaskExt（extern_trait）的应用。

3.1.2 内存管理

【待完善】 简述 mmap/munmap/brk 等的实现，涵盖匿名映射、文件映射、懒分配、CoW 等机制。

3.1.3 其他系统调用

【待完善】 一段话概括文件 I/O、网络、同步（futex）等模块的实现思路， 说明遵循 Linux 语义实现了 200+ 个系统调用，不逐一展开。

3.2 文件系统

【待完善】介绍 VFS 层设计（支持物理 FS 与伪 FS 统一抽象，参考 Linux 设计）。重点说明页缓存、块缓存、目录项缓存三层协同加速的设计。

3.3 网络、终端与其他子系统

【待完善】 一段话带过：网络栈基于 smoltcp，支持 TCP/UDP/UDS 及多设备路由； 终端支持 tty/pty、line discipline、job control；实现思路与主流兼容内核无本质差别。

3.4 信号机制

【待完善】 说明信号投递时机（系统调用返回前检查）、与异步机制的交互（信号可中断阻塞的系统调用），以及对 SA_RESTART 等语义的处理。

3.5 多级缓存优化

【待完善】 展开说明页缓存（page cache）、块缓存（block cache）、目录项缓存（dcache）三层缓存的实现细节，以及脏页写回策略。

3.6 多核调度优化

【待完善】说明当前调度实现的基础（基于 ArceOS axtask）， 以及针对多核场景规划/已实施的优化（负载均衡、调度延迟等）。如工作尚未完成，说明设计方向和参考（KRR, OSDI’25）。

3.7 虚拟内存优化

【待完善】说明当前虚拟内存实现（VMA + 页表双层结构），以及参考 CortenMM（SOSP’25）取消独立 VMA 抽象的优化方向。 如工作尚未完成，说明设计思路。

第 4 章 AI 应用支持

【待完善】开头一段：说明 AIoT 场景下操作系统对 AI 应用支持的需求背景，以及本章工作的定位（探索性/规划性内容，目前处于进行中）。

4.1 目标平台与硬件环境

【待完善】简述目标开发板（配备 NPU 的边缘计算设备），说明需要完成的移植工作（Starry OS 跨架构支持、驱动适配）。

4.2 NPU 驱动设计

【待完善】简述 NPU 驱动开发的思路和挑战： 驱动模型、与内核内存管理的交互、DMA 支持等。

4.3 AI 应用运行支持

【待完善】 简述目标应用（YOLO 目标检测、dora-rs 数据流框架）的运行需求，以及 Starry OS 需要提供的支持（推理框架兼容性、实时调度等）。

【待完善】 如果工作尚未完成，可在此说明当前进展及后续计划。

第 5 章 实验评测

【待完善】开头一段：说明本章从功能完善性和性能两个维度对 StarryOS 进行评测， 包括操作系统大赛标准测例、应用兼容性验证，以及性能基准测试。

5.1 实验环境

【待完善】简述测试所用的硬件平台（x86/riscv64/aarch64 等架构、QEMU 版本）和软件环境（Rust 工具链版本、Alpine Linux 镜像版本等）。

5.2 功能测试

5.2.1 系统调用测例

【待完善】 介绍操作系统大赛的标准测例套件（参考 thuthesis-zyj 表 5.1 的套件说明）， 说明 StarryOS 的通过情况（可配表格，对比赛道测例通过率）。

5.2.2 应用兼容性测试

【待完善】 展示在 Alpine Linux 下运行实际应用的验证结果，包括：

- 开发工具链（GCC 编译、Cargo 构建、Git 操作）
- 图形应用（X11/dwm + ImageMagick 等）
- 其他典型应用（Python、SQLite、Redis、xv6 嵌套运行等）

可用截图或描述性文字。

5.3 性能测试

【待完善】 说明性能测试的方法和基准（lmbench、UnixBench 或自定义 microbenchmark）。

5.3.1 系统调用开销

【待完善】 测试典型系统调用（getpid、read、write 等）的延迟， 与 Linux 基线对比。

5.3.2 I/O 性能

【待完善】测试文件读写吞吐量，体现异步机制和多级缓存优化的效果。

5.4 开发工作量统计

【待完善】统计整体工作量，参考 thuthesis-zyj 表 5.3 的形式： 主仓库新增代码行数（约 6.5 万行）、子仓库数量（33+）、commit 数（约 1000）等。

第 6 章 总结与展望

6.1 工作总结

【待完善】3-4 段，逐一总结各章的主要工作和成果：

1. 组件化设计探索（extern_trait、用户态切换）
2. 兼容性内核实现（200+ syscall、Alpine 生态）
3. 性能优化（访存、缓冲区、异步 I/O、缓存）
4. AI 应用支持探索

6.2 未来工作展望

【待完善】2-3 段，说明后续计划：

- 多核调度优化（参考 KRR 等工作）
- 虚拟内存优化（参考 CortenMM，取消 VMA 抽象）
- NPU 驱动完善与 AI 应用落地
- 组件进一步开源复用（已有 weak-map、extern-trait、scope-local 等发布计划）

参考文献

[1] KLABNIK S, NICHOLS C. The Rust Programming Language[M]. No Starch Press, 2023.

[2] PANTER S, EISTY N. Rusty Linux: Advances in Rust for Linux Kernel Development[C]// Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). 2024.

[3] LI Z, OTHERS. Rust for Linux: Understanding the Security Impact of Rust in the Linux Kernel[C]//2024 Annual Computer Security Applications Conference (ACSAC). 2024.

[4] CUTLER C, KAASHOEK M F, MORRIS R T. The Benefits and Costs of Writing a POSIX Kernel in a High-Level Language[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018.

[5] LANKES S, PICKARTZ S, BREITBART J. HermitCore: A Unikernel for Extreme Scale Computing[C]//Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers. 2016.

[6] LANKES S, OTHERS. RustyHermit: A Scalable, Rust-Based Virtual Execution Environment[C]//International Conference on High Performance Computing. Cham: Springer International Publishing, 2020.

[7] SHARMA A, OTHERS. Rust for Embedded Systems: Current State and Open Problems[C]// Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024.

[8] LEVY A, OTHERS. Ownership is Theft: Experiences Building an Embedded OS in Rust[C]// Proceedings of the 8th Workshop on Programming Languages and Operating Systems. 2015.

[9] LEVY A, OTHERS. The Case for Writing a Kernel in Rust[C]//Proceedings of the 8th Asia-Pacific Workshop on Systems. 2017.

[10] LANKES S, BREITBART J, PICKARTZ S. Exploring Rust for Unikernel Development[C]// Proceedings of the 10th Workshop on Programming Languages and Operating Systems. 2019.

[11] KANTEE A. Flexible Operating System Internals: The Design and Implementation of the Anykernel and Rump Kernels[D]. 2012.

[12] PENG Y, OTHERS. Framekernel: A Safe and Efficient Kernel Architecture via Rust-based Intra-kernel Privilege Separation[C]//Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems. 2024.

[13] TIAN H, OTHERS. Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB[Z]. 2025.

[14] ZHANG J, OTHERS. CortenMM: Efficient Memory Management with Strong Correctness Guarantees[C]//Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP). 2025.

附录 A 外文资料的调研阅读报告（或书面翻译）

调研阅读报告题目（或书面翻译题目）

写出至少 5000 外文印刷字符的调研阅读报告或者书面翻译 1-2 篇（不少于 2 万外文印刷符）。

A.1 Lorem ipsum dolor.

A.1.1 Lorem ipsum dolor.

参考文献（或书面翻译对应的原文索引）

[1] 某某某. 信息技术与信息服务国际研讨会论文集: A 集［C］北京：中国社会科学出版社，1994.

附录 B 其他附录内容

【待完善】填写其他附录内容（如有）

致　谢

【待完善】 填写致谢内容

声　明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____

综合论文训练记录表

<table>
<tr><td>学生姓名</td><td>王铮</td><td>学号</td><td>2022010862</td><td>班级</td><td>计 26</td></tr>
<tr><td>论文题目</td><td colspan="5">AIoT 操作系统性能优化关键技术研究</td></tr>
<tr><td>主要内容以及进度安排</td><td colspan="5">主要内容：

进度安排：

指导教师签字：

考核组组长签字：

年　月　日</td></tr>
<tr><td>中期考核意见</td><td colspan="5">考核组组长签字：

年　月　日</td></tr>
</table>

指导教师评语	指导教师签字： 年　月　日
评阅教师评语	评阅教师签字： 年　月　日
答辩小组评语	答辩小组组长签字： 年　月　日

总成绩：_____

教学负责人签字：_____

年　月　日